

Module 15 – Advanced Python Programming

Expanded Theory & Practical Answers

1. Printing on Screen

Theory:

- 1 The print() function is the primary way to display output in Python programs. It helps programmers verify results and communicate with users.
- 2 Python supports multiple formatting techniques including comma separation, format(), and f-strings. Among these, f-strings are preferred because they are readable and efficient.
- 3 Formatted output is important in professional programs to ensure clarity, especially when displaying numbers, variables, and aligned data.
- 4 The print() function can also control separators, line endings, and formatting precision.

Practical Code:

```
name = "Vijay"
marks = 92.456

print("Basic print:", name, marks)
print("Formatted using format(): {:.2f}".format(marks))
print(f"Using f-string -> Name: {name}, Marks: {marks:.2f}")
```

2. Reading Data from Keyboard

Theory:

- 1 The input() function allows interactive programs by accepting user data at runtime.
- 2 All input is received as text, so explicit type conversion is required for numbers using int() or float().
- 3 Validating user input prevents runtime errors and improves reliability.
- 4 Interactive input is widely used in calculators, login systems, and data entry programs.

Practical Code:

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
height = float(input("Enter your height: "))

print(f"User -> {name}, Age: {age}, Height: {height}")
```

3. Opening and Closing Files

Theory:

- 1 File handling allows persistent storage of data beyond program execution.

- 2 Different modes such as read (r), write (w), append (a), and read/write (r+) control file access.
- 3 Always closing files prevents memory leaks and data corruption.
- 4 Using 'with open()' is recommended because it automatically handles closing.

Practical Code:

```
with open("sample.txt", "w") as file:
    file.write("Learning Advanced Python File Handling")

print("File written successfully!")
```

4. Reading and Writing Files

Theory:

- 1 read() loads entire file content, while readline() reads line-by-line.
- 2 write() inserts text into files, and writelines() writes multiple lines.
- 3 tell() returns the cursor position, useful for tracking file navigation.
- 4 Efficient file reading is important when handling large datasets.

Practical Code:

```
with open("sample.txt", "r") as file:
    content = file.read()
    print("File Content:")
    print(content)
```

5. Exception Handling

Theory:

- 1 Exceptions occur when unexpected situations arise during execution.
- 2 try-except blocks allow programs to recover gracefully instead of crashing.
- 3 Multiple exception blocks can handle different error types.
- 4 finally ensures cleanup operations always execute.

Practical Code:

```
try:
    a = int(input("Enter number: "))
    result = 100 / a
    print("Result:", result)

except ZeroDivisionError:
    print("Division by zero is not allowed")

except ValueError:
    print("Invalid numeric input")

finally:
    print("Execution complete")
```

6. Classes and Objects

Theory:

- 1 Classes define templates for creating objects with attributes and behaviors.
- 2 Objects represent real-world entities in software.
- 3 Encapsulation groups related data and functions.
- 4 OOP improves modularity, reuse, and maintainability.

Practical Code:

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def display(self):
        print(f"Student: {self.name}, Grade: {self.grade}")

s = Student("Vijay", "A")
s.display()
```

7. Inheritance

Theory:

- 1 Inheritance allows child classes to reuse parent functionality.
- 2 It supports hierarchical design and reduces duplicate code.
- 3 `super()` enables access to parent methods.
- 4 Useful for extending features in scalable applications.

Practical Code:

```
class Person:
    def show(self):
        print("I am a person")

class Student(Person):
    def show_student(self):
        print("I am a student")

s = Student()
s.show()
s.show_student()
```

8. Method Overloading and Overriding

Theory:

- 1 Method overloading simulates multiple behaviors using default arguments.
- 2 Method overriding replaces parent functionality in child classes.
- 3 These concepts increase flexibility and adaptability.
- 4 Widely used in polymorphic system design.

Practical Code:

```
class Parent:
    def display(self):
        print("Parent version")

class Child(Parent):
    def display(self):
        print("Child overridden version")

obj = Child()
obj.display()
```

9. SQLite Database Connectivity

Theory:

- 1 SQLite is a lightweight embedded database engine.
- 2 Python uses sqlite3 to execute SQL commands.
- 3 Supports persistent storage without server setup.
- 4 Ideal for small applications and prototypes.

Practical Code:

```
import sqlite3

conn = sqlite3.connect("students.db")
cur = conn.cursor()

cur.execute("CREATE TABLE IF NOT EXISTS student(name TEXT)")
cur.execute("INSERT INTO student VALUES('Vijay')")

conn.commit()

for row in cur.execute("SELECT * FROM student"):
    print(row)

conn.close()
```

10. Integrated Mini Program

Theory:

- 1 Combines input handling, exception safety, and logic.
- 2 Demonstrates real-world programming structure.
- 3 Encourages modular thinking and testing.

4 Shows how multiple Python features work together.

Practical Code:

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))

    print("Addition:", a + b)
    print("Subtraction:", a - b)

except ValueError:
    print("Please enter valid integers")

finally:
    print("Calculator session ended")
```